

Apunts javascript

ARRAYS

Array.from()

Crea una nova instància d'Array a partir d'un objecte iterable i Pot acceptar una funció de mapeig opcional per transformar cada element

Exemple: `Array.from('foo') // ["f", "o", "o"]`

Array.prototype.pop()

Elimina l'últim element d'un array i el retorna, modifica la longitud de l'array original i retorna undefined si l'array està buit.

Exemple: `['broccoli', 'cauliflower', 'tomato'].pop() // "tomato"`

Array.prototype.push()

Afegeix un o més elements al final d'un array, retorna la nova longitud de l'array i és molt pràctic per afegir valors a un array

Exemple: `['pigs', 'goats'].push('cows') // 3`

Array.prototype.slice()

Retorna una còpia superficial d'una porció de l'array, **no modifica l'array original** i accepta índexs negatius (compten des del final).

Exemple: `['ant', 'bison', 'camel', 'duck'].slice(2) // ["camel", "duck"]`

Array.prototype.sort()

Ordena els elements d'un array localment, **Per defecte:** ordena convertint elements a strings segons valors Unicode, per ordenar números: `arr.sort((a, b) => a - b)` i per caràcters no ASCII: usar `String.localeCompare()`.

Accepta una funció de comparació opcional: `compareFunction(a, b)`

- Si retorna `< 0`: a va abans que b
- Si retorna `0`: a i b queden sense canvis
- Si retorna `> 0`: b va abans que a

Exemple: `[40, 1, 5, 200].sort((a, b) => a - b) // [1, 5, 40, 200]`

DATE

Date.now()

Retorna els mil·lisegunds transcorreguts des de l'1 de gener de 1970 (00:00:00 UTC) i útil per crear registres temporals o identificadors únics

Exemple: `Date.now() // 1698235800000`

Date.prototype.getDate()

Retorna el dia del mes (1-31) segons l'hora local

Exemple: `new Date('December 25, 1995').getDate() // 25`

Date.prototype.getDay()

Retorna el dia de la setmana (0-6) on 0 és diumenge

Exemple: new Date('December 25, 1995').getDay() // 1 (Dilluns)

Date.prototype.getFullYear()

Retorna l'any complet (quatre dígits) segons l'hora local i utilitza aquest mètode en lloc de getYear()

Exemple: new Date('July 20, 69').getFullYear() // 1969

Date.prototype.getHours()

Retorna l'hora (0-23) segons l'hora local

Exemple: new Date('December 25, 1995 23:15:30').getHours() // 23

Date.prototype.getMinutes()

Retorna els minuts (0-59) segons l'hora local

Exemple: new Date('December 25, 1995 23:15:30').getMinutes() // 15

Date.prototype.getMonth()

Retorna el mes (0-11) on 0 és gener i 11 és desembre

Exemple: new Date('December 25, 2014').getMonth() // 11

Date.prototype.getSeconds()

Retorna els segons (0-59) segons l'hora local

Exemple: new Date('July 20, 69 00:20:18').getSeconds() // 18

Date.prototype.getTime()

Retorna el valor numèric en mil·lisegons des de l'1 de gener de 1970 i útil per copiar dates o mesurar temps d'execució

Exemple: new Date(1994, 12, 10).getTime() // 788918400000

Date.prototype.toString()

Retorna la porció de data en format llegible en anglès (Dia Mes Data Any)

Exemple: new Date(1993, 5, 28).toString() // "Wed Jun 28 1993"

Date.prototype.toLocaleDateString()

Retorna una representació de la data sensible al idioma/localització i accepta arguments locals i options per personalitzar el format

Exemple: new Date().toLocaleDateString('de-DE') // "20.12.2012"

Date.prototype.toLocaleTimeString()

Retorna una representació de l'hora sensible al idioma/localització i accepta arguments locals i options per personalitzar el format

Exemple: new Date('August 19, 1975 23:15:30').toLocaleTimeString('en-US') // "1:15:30 AM"

DO...WHILE

do...while

Crea un bucle que executa una sentència especificada fins que la condició s'avalua com a falsa, **la condició s'avalua DESPRÉS d'executar la sentència** i això garanteix que la sentència **s'executa almenys una vegada**

Exemple:

```
let i = 0;
do {
  i += 1;
  console.log(i);
} while (i < 5);
// Output: 1 2 3 4 5
```

DOCUMENT

Document.createElement()

Crea un element HTML especificat pel tagName, en documents HTML, converteix tagName a minúscules abans de crear l'element, retorna un nou Element (o HTMLElement en documents HTML).

Sintaxi: document.createElement(tagName, options), on els Paràmetres:

- tagName: string que especifica el tipus d'element a crear
- options (opcional): objecte amb propietats com is per custom elements

Exemple:

```
const newDiv = document.createElement("div");
const newContent = document.createTextNode("Hola!");
newDiv.appendChild(newContent);
document.body.appendChild(newDiv);
```

Document.createTextNode()

Crea un nou node de text, aquest mètode pot ser usat per escapar caràcters HTML i retorna un node Text.

Sintaxi: document.createTextNode(data), on el paràmetre:

- data: cadena de text que conté les dades per posar al node de text

Exemple:

```
var newtext = document.createTextNode("YES! ");
document.getElementById("p1").appendChild(newtext);
```

Document.querySelector()

Retorna el **primer element** del document que coincideix amb el selector CSS especificat i utilitza un recorregut en profunditat (depth-first) dels nodes del document.

Sintaxi: document.querySelector(selectores), on el paràmetre:

- selectors: cadena que conté un o més selectors CSS separats per comes

Exemple:

```
var el = document.querySelector(".miClase");
```

// O més complex:

```
var el = document.querySelector("div.user-panel input[name='login']");
```

Document.querySelectorAll()

Retorna una NodeList **estàtica** (no viva) amb **tots els elements** del document que coincideixen amb els selectors, els caràcters especials que no són part de la sintaxi CSS estàndard s'han d'escapar amb barra invertida i retorna una NodeList amb tots els elements coincidents

Sintaxi: document.querySelectorAll(selectors), on el paràmetre:

- selectors: cadena CSS vàlida amb un o més selectors separats per comes

Exemple:

```
var elementList = document.querySelectorAll("div.note, div.alert");
```

ELEMENT

element.addEventListener()

Registra un esdeveniment a un objecte específic i pots registrar múltiples listeners cridant addEventListener() amb diferents tipus d'esdeveniments

Sintaxi: target.addEventListener(tipo, listener, useCapture), on els paràmetres:

- tipo: cadena representant el tipus d'esdeveniment (click, mousedown, etc.)
- listener: objecte que rep la notificació o una funció JavaScript
- useCapture (opcional): si és true, inicia la captura de l'esdeveniment

Exemple:

```
var el = document.getElementById("t");el.addEventListener("click", modifyText, false);
```

Element.after()

Insereix un conjunt de Nodes o strings després de l'element i els strings s'insereixen com a nodes Text equivalents

Sintaxi: element.after(node1, node2, ..., nodeN)

Exemple:

```
let p = document.createElement("p");  
let span = document.createElement("span");  
p.after(span, "Text"); // Insereix span i text després de p
```

Element.getAttribute()

Retorna el valor d'un atribut especificat de l'element, retorna null o "" si l'atribut no existeix i en elements HTML converteix l'attributeName a minúscules.

Sintaxi: element.getAttribute(attributeName)

Exemple:

```
const div1 = document.getElementById('div1');  
const exampleAttr = div1.getAttribute('id'); // "div1"
```

Element.innerHTML

Obté o estableix el marcatge HTML o XML contingut dins de l'element, **RISC DE SEGURETAT:** Pot ser un vector d'atac si s'insereix contingut d'usuaris i establir innerHTML elimina tots els descendents de l'element i els substitueix

Exemple:

```
const logElem = document.querySelector(".log");  
logElem.innerHTML += "Nou contingut<br/>";
```

Element.insertAdjacentElement()

Insereix un node element en una posició relativa a l'element, les posicions: 'beforebegin', 'afterbegin', 'beforeend', 'afterend'.

Sintaxi: element.insertAdjacentElement(position, element)

Exemple:

```
activeElem.insertAdjacentElement('beforebegin', tempDiv);
```

Element.insertAdjacentHTML()

Parseja text com HTML/XML i insereix els nodes resultants al DOM, **Més ràpid que innerHTML** perquè no reparseja l'element sencer i Posicions: "beforebegin", "afterbegin", "beforeend", "afterend"

Sintaxi: element.insertAdjacentHTML(position, text)

Exemple:

```
subject.insertAdjacentHTML('afterbegin', '<strong>text</strong>');
```

Element.insertAdjacentText()

Insereix un nou node de text en una posició relativa, similar a insertAdjacentHTML però per text pla (no HTML)

Sintaxi: element.insertAdjacentText(where, data)

Exemple:

```
para.insertAdjacentText('afterbegin', textInput.value);
```

Element.prepend()

Insereix Nodes o strings **abans** del primer fill de l'element i els strings s'insereixen com a nodes Text

Sintaxi: element.prepend(param1, param2, ..., paramN)

Exemple:

```
div.prepend("Headline: ", p); // Insereix text i element al principi
```

Element.remove()

Elimina un element del DOM i no té paràmetres ni retorna cap valor

Sintaxi: element.remove()

Exemple:

```
const element = document.getElementById("div-02");element.remove(); // Elimina el div
```

Element.removeAttribute()

Elimina l'atribut amb el nom especificat de l'element i no genera error si l'atribut no existeix

Sintaxi: element.removeAttribute(attrName)

Exemple:

```
document.getElementById("div1").removeAttribute("align");
```

Element.setAttribute()

Estableix el valor d'un atribut en l'element, si l'atribut existeix s'actualitza i si no s'afegeix, en elements HTML el nom s'auto-converteix a minúscules i per atributs booleans, qualsevol valor significa true (recomanat: "" o el nom de l'atribut)

Sintaxi: `element.setAttribute(name, value)`, on els paràmetres:

- `name`: nom de l'atribut
- `value`: valor a assignar (es converteix a string automàticament)

Exemple:

```
button.setAttribute("name", "helloButton");button.setAttribute("disabled", ""); // Deshabilita el botó
```

Visualització de posicions per `insertAdjacent*`:

```
<!-- beforebegin -->
<p>
  <!-- afterbegin -->
  contingut
  <!-- beforeend -->
</p>
<!-- afterend -->
```

FOR

for

Crea un bucle amb tres expressions opcionals entre parèntesis, separades per punt i coma i per executar múltiples sentències, usar un bloc `{ ... }`

Sintaxi: `for ([expresion-inicial]; [condicion]; [expresion-final]) sentència`, on els components:

- `expresion-inicial`: declaració de variable o expressió, típicament usada com a comptador
- `condicion`: avaluada abans de cada iteració; si és `true`, s'executa la sentència
- `expresion-final`: avaluada al final de cada iteració, generalment per incrementar el comptador

Exemple:

```
for (var i = 0; i < 9; i++) { n += i; mifuncion(n);}
```

TEMPLATE

Template Literals (Plantilles literals)

Permeten incrustar expressions JavaScript directament dins strings, útils per generar HTML markup dinàmic i queries SQL i permeten strings multilínia sense necessitat de concatenació

Sintaxi: utilitzant backticks ``` i `${}` per a expressions

Exemple:

```
const user = { name: "Marie", age: 25 };
const userProfile = ` <div>   <h2>Name: ${user.name}</h2> <p>Age: ${user.age}</p> </div>`;
```

HTML INNER TEXT

HTML `Element.innerText`

Representa el contingut de text **renderitzat** d'un node i els seus descendents, Com a getter: aproxima el text que l'usuari obtindria si seleccionés i copiés el contingut, Com a setter: substitueix els fills de l'element amb el valor donat, convertint salts de línia en `<br>` i és conscient de l'aparença renderitzada (ignora elements ocults, respecta `<br>`, etc.)

Diferència amb `textContent`: `innerText` és conscient del renderitzat, `textContent` no ho és

Exemple:

```
const source = document.getElementById("source");
console.log(source.innerText); // Text visible renderitzat
```

IF ELSE

if...else

Executa una sentència si una condició específica s'avalua com a true, si és false s'executa la sentència de la clàusula else opcional, Es poden anidar múltiples sentències if...else per crear clàusules else if, **Bona pràctica**: sempre usar blocs { } per evitar comportaments inesperats i **Valors truthy/falsy**: undefined, null, 0, -0, NaN, "" s'avaluen com a false

Sintaxi:

```
if (condició)
    sentència1
else
    sentència2
```

Exemple:

```
if (x > 50) { console.log("Major que 50");} else if (x > 5) { console.log("Major que 5");} else
{ console.log("5 o menys");}
```

JSON

JSON.parse()

Analitza una cadena de text com a JSON i retorna l'objecte JavaScript corresponent, llança una excepció SyntaxError si el text no és JSON vàlid.

Sintaxi: JSON.parse(text[, reviver]) on els paràmetres:

- text: cadena que es convertirà a JSON
- reviver (opcional): funció que transforma el valor abans de retornar-lo

Exemple:

```
JSON.parse('{ "x": 5, "y": 6 }'); // {x: 5, y: 6}
JSON.parse('[1, 5, "false"]'); // [1, 5, "false"]
JSON.parse('true'); // true
```

JSON.stringify()

Converteix un objecte o valor JavaScript en una cadena de text JSON

Sintaxi: JSON.stringify(value[, replacer[, space]]), on els paràmetres:

- value: valor a convertir a JSON
- replacer (opcional): funció o array que altera el procés de conversió
- space (opcional): controla la indentació per fer el JSON més llegible

Exemple:

```
JSON.stringify({x: 5, y: 6}); // '{"x":5,"y":6}'
JSON.stringify([1, 2, 3]); // '[1,2,3]'
```

Resum de diferències clau:

- **innerText vs textContent**: innerText respecta el renderitzat (CSS, visibilitat), textContent retorna tot el text
- **JSON.parse() vs JSON.stringify()**: parse converteix string JSON → objecte JS; stringify converteix objecte JS → string JSON

MATH

Math.ceil()

Arrodoneix **cap amunt** i retorna el **menor enter que és $\geq$ al nombre donat**.

Exemple:

```
console.log(Math.ceil(7.004)); // 8
```

Math.floor()

Arrodoneix **cap avall** i retorna el **major enter que és $\leq$ al nombre donat**.

Exemple:

```
console.log(Math.floor(5.95)); // 5
```

Math.random()

Retorna un nombre **pseudoaleatori** entre **0 (inclòs)** i **1 (exclòs)**, no és adequat per a ús criptogràfic.

Exemple:

```
console.log(Math.random()); // nombre entre 0 i <1
```

Math.round()

Retorna el valor d'un número arrodonit a l'enter més proper

Sintaxi: `Math.round(x)`, on els paràmetres:

- x: un número

Comportament:

- Si la part fraccionària és **major que 0.5**: s'arrodoneix a l'enter superior
- Si la part fraccionària és **menor que 0.5**: s'arrodoneix a l'enter inferior
- Si la part fraccionària és **exactament 0.5**: s'arrodoneix cap a $+\infty$ (cap amunt en positius, menys negatiu en negatius)

Exemple:

```
Math.round(0.9); // 1
Math.round(5.95); // 6
Math.round(5.5); // 6
Math.round(5.05); // 5
Math.round(-5.05); // -5
Math.round(-5.5); // -5 (cap a  $+\infty$ )
Math.round(-5.95); // -6
```

Nota important: Quan la part fraccionària és exactament 0.5, `Math.round()` arrodoneix en la direcció de $+\infty$, cosa que pot diferir d'altres llenguatges de programació que arrodoneixen allunyant-se del zero.

NODE

Node.hasChildNodes()

Retorna un valor booleà indicant si el Node donat té nodes fills o no i no té paràmetres

Sintaxi: `node.hasChildNodes()`

Exemple:

```
let foo = document.getElementById('foo');if (foo.hasChildNodes()) { // Fer alguna cosa amb 'foo.childNodes'}
```


Node.insertBefore()

Insereix un node abans del node de referència com a fill d'un node pare indicat, si el node fill ja existeix en el document, insertBefore() el mou de la posició actual a la nova posició, en canvi si el node de referència és null, el node indicat s'afegeix al final de la llista de fills .

Sintaxi: parentNode.insertBefore(newNode, referenceNode), on els paràmetres:

- newNode: el node a inserir
- referenceNode: el node abans del qual s'insereix newNode (no és opcional, cal passar explícitament un Node o null)

Exemple:

```
var sp1 = document.createElement("span");
var sp2 = document.getElementById("childElement");
var parentDiv = sp2.parentNode;parentDiv.insertBefore(sp1, sp2); // Insereix sp1 abans de sp2
```

Truc per insertAfter: parentDiv.insertBefore(sp1, sp2.nextSibling);

Node.appendChild()

Afegeix un nou node al **final** de la llista de fills d'un element pare especificat, si el fill (child) és una referència a un node existent en el document, es treu del pare actual per ser posat en el nou node pare.

Sintaxi: element.appendChild(aChild), on el paràmetre:

- aChild: el node a adjuntar al node pare (normalment un element)

Exemple:

```
var p = document.createElement("p");
document.body.appendChild(p); // Afegeix p al final del body
```

Notes importants:

- **insertBefore** vs **appendChild**: insertBefore permet especificar la posició exacta, appendChild sempre afegeix al final
- Si el node ja existeix en el document, aquests mètodes el **mouen** (no cal eliminar-lo primer)
- Per fer una còpia abans d'inserir, usar Node.cloneNode()
- **Nova API disponible:** ParentNode.append() suporta múltiples arguments i cadenes de text

NUMBER

Number.isInteger()

Determina si el valor passat és un enter (integer), Valor de retorn: true si el valor és un enter, false en cas contrari, Si el valor és NaN o Infinity, retorna false i També retorna true per nombres de punt flotant que poden representar-se com a enters

Sintaxi: Number.isInteger(value), on el paràmetre:

- value: el valor a comprovar si és un enter

Exemple:

```
Number.isInteger(5);    // true
Number.isInteger(5.0);  // true
Number.isInteger(5.5);  // false
Number.isInteger("5");  // false
Number.isInteger(NaN);  // false
```

Number.isNaN()

Determina si el valor passat és NaN i el seu tipus és Number, És una versió més robusta que la funció global isNaN() i Valor de retorn: true si el valor és un número amb valor NaN, false en cas contrari.

Sintaxi: Number.isNaN(value), on el paràmetre:

- value: el valor a comprovar si és NaN

Exemple:

```
Number.isNaN(NaN);      // true
Number.isNaN(Number.NaN); // true
Number.isNaN(0 / 0);    // true
Number.isNaN("NaN");    // false (és un string)
Number.isNaN(undefined); // false
```

Number.parseFloat()

Parseja un argument i retorna un número de punt flotant, si no es pot parsejar un número, retorna NaN, el Valor de retorn: Un número de punt flotant parseiat del string donat, o NaN si no es pot convertir i Té la mateixa funcionalitat que la funció global parseFloat()

Sintaxi: Number.parseFloat(string), on el paràmetre:

- string: el valor a parsejar, convertit a string. Els espais en blanc inicials s'ignoren

Exemple:

```
Number.parseFloat("3.14");           // 3.14
Number.parseFloat("3.14more");       // NaN
Number.parseFloat(" 3.14");          // 3.14
Number.parseFloat("abc");             // NaN
```

Number.parseInt()

Parseja un argument string i retorna un enter de la base (radix) especificada i valor de retorn: Un enter parseiat del string, o NaN si no es pot parsejar

Sintaxi: Number.parseInt(string, radix), on els paràmetres:

- string: el valor a parsejar, convertit a string. Els espais inicials s'ignoren
- radix (opcional): un enter entre 2 i 36 que representa la base del sistema numèric

Exemple:

```
Number.parseInt("10");      // 10
Number.parseInt("10", 10);  // 10 (decimal)
Number.parseInt("10", 2);   // 2 (binari)
Number.parseInt("0xF", 16); // 15 (hexadecimal)
Number.parseInt("321", 2);  // NaN (no vàlid en binari)
```

Number.prototype.toFixed()

Formata un número utilitzant notació de punt fix, el Valor de retorn: Un **string** representant el número amb notació de punt fix i Excepció: RangeError si digits és massa petit o massa gran

Sintaxi: number.toFixed(digits), on el paràmetre:

- digits (opcional): el nombre de dígitos a mostrar després del punt decimal (entre 0 i 20). Si s'omet, s'usa 0

Exemple:

```
(123.456).toFixed(2); // "123.46"
(123.456).toFixed(0); // "123"
(0.004).toFixed(2);   // "0.00"
(1.23e+5).toFixed(2); // "123000.00"
```

parseInt()

Converteix (parseja) un argument de tipus cadena i retorna un enter de la base especificada i eé una funció de alt nivell no associada a cap objecte.

Sintaxi: parseInt(cadena, base), on els paràmetres:

- cadena: una cadena que representa el valor a convertir
- base: un enter que representa la base (radix) del sistema numèric

Comportament:

- Comprova el primer argument i intenta retornar un enter de la base especificada
- Si troba un caràcter que no és numeral de la base, l'ignora i tots els següents
- parseInt trunca els números a valors enters
- Si el primer caràcter no es pot convertir en número, retorna NaN

Si no s'especifica la base o és 0:

- Si comença per "0x": base 16 (hexadecimal)
- Si comença per "0": base 8 (octal) - desaconsellat
- Altrament: base 10 (decimal)

Exemples:

```
parseInt("F", 16);    // 15
parseInt("17", 8);    // 15
parseInt("15", 10);   // 15
parseInt("Hello", 8); // NaN
parseInt("0x11", 16); // 17
parseInt("546", 2);   // NaN (no vàlid en binari)
```

Diferències clau:

- **Number.isNaN()** vs **isNaN()**: Number.isNaN() és més estricta (només retorna true per NaN de tipus Number)
- **parseFloat/parseInt**: parseFloat retorna decimals, parseInt només enters
- **toFixed()**: retorna un **string**, no un número
- **isInteger()**: també accepta flotants que representen enters (ex: 5.0 retorna true)

EVENTS

Referència d'Esdeveniments

Els esdeveniments notifiquen al codi sobre coses interessants que han passat. Cada esdeveniment està representat per un objecte basat en la interfície Event.

Esdeveniments Estàndard més comuns:

Esdeveniments de Mouse:

- click: Un botó de dispositiu apuntador s'ha premut i soltat sobre un element
- dblclick: Doble clic sobre un element
- mousedown: Botó premut sobre un element
- mouseup: Botó alliberat sobre un element
- mousemove: El punter es mou sobre un element
- mouseenter: El punter entra en un element
- mouseleave: El punter surt d'un element

Esdeveniments de Teclat:

- keydown: Una tecla es prem
- keypress: Una tecla es prem i normalment produeix un caràcter
- keyup: Una tecla s'allibera

Esdeveniments de Formulari:

- submit: Un formulari s'envia
- change: Un element ha perdut el focus i el seu valor ha canviat
- input: El valor d'un element s'ha modificat
- focus: Un element rep el focus
- blur: Un element perd el focus

Esdeveniments de Document:

- DOMContentLoaded: El document ha acabat de carregar (però no recursos dependents)
- load: Un recurs i els seus recursos dependents han acabat de carregar

Esdeveniments de Mitjans (Media):

- play: La reproducció ha començat
- pause: La reproducció s'ha pausat
- ended: La reproducció ha arribat al final

Shuffle

Shuffle an Array (Barrejar un array)

Solució incorrecta (biaix estadístic):

```
function shuffle(array) {  
    array.sort(() => Math.random() - 0.5);  
}
```

Problema: No totes les permutacions tenen la mateixa probabilitat. El mètode sort() no està pensat per ser usat així.

Solució correcta - Algorisme Fisher-Yates:

```
function shuffle(array) {  
    for (let i = array.length - 1; i > 0; i--) {  
        let j = Math.floor(Math.random() * (i + 1));  
        // Intercanviar elements array[i] i array[j]  
        [array[i], array[j]] = [array[j], array[i]];  
    }  
}
```

Com funciona Fisher-Yates:

1. Recorre l'array en ordre invers
2. Per cada element, l'intercanvia amb un element aleatori anterior (o ell mateix)
3. Garanteix que totes les permutacions tinguin la mateixa probabilitat
4. Millor rendiment que usar sort()

Exemple d'ús:

```
let arr = [1, 2, 3];  
shuffle(arr);  
console.log(arr); // [3, 2, 1] o qualsevol altra permutació
```

Avantatges de Fisher-Yates:

- Totes les permutacions tenen probabilitat igual
- Millor rendiment (no hi ha overhead de sorting)
- Algorisme provadament correcte

STRING

String.prototype.charAt()

Retorna un nou string consistent en la unitat UTF-16 única localitzada a l'offset especificat, Valor de retorn: String representant el caràcter a l'índex especificat. Si l'índex està fora de rang, retorna un string buit i els caràcters s'indexen d'esquerra a dreta començant per 0

Sintaxi: `string.charAt(index)`, on el paràmetre:

- `index`: enter entre 0 i `str.length - 1`. Per defecte és 0

Example:

```
const sentence = 'The quick brown fox';
sentence.charAt(4); // "q"
sentence.charAt(0); // "T"
```

String.prototype.endsWith()

Determina si un string acaba amb els caràcters d'un string especificat i Valor de retorn: `true` si es troben els caràcters al final, `false` altrament

Sintaxi: `string.endsWith(searchString, endPosition)`, on els paràmetres:

- `searchString`: els caràcters a buscar al final. No pot ser regex
- `endPosition` (opcional): la posició final on s'espera trobar `searchString`. Per defecte `str.length`

Example:

```
const str1 = 'Cats are the best!';
str1.endsWith('best!'); // true
str1.endsWith('best', 17); // true
str1.endsWith('Cats'); // false
```

String.prototype.includes()

Fa una cerca sensible a majúscules/minúscules per determinar si un string pot trobar-se dins d'un altre i Valor de retorn: `true` si el string de cerca es troba, `false` altrament

Sintaxi: `string.includes(searchString, position)`, on els paràmetres:

- `searchString`: string a buscar. No pot ser regex
- `position` (opcional): posició on començar la cerca. Per defecte 0

Example:

```
const sentence = 'The quick brown fox jumps';
sentence.includes('fox'); // true
sentence.includes('Fox'); // false
(case-sensitive)sentence.includes('fox', 20); // false
```

String.prototype.indexOf()

Cerca l'string complet i retorna l'índex de la **primera ocurrència** del substring especificat i Valor de retorn: L'índex de la primera ocurrència, o **-1** si no es troba

Sintaxi: `string.indexOf(searchString, position)`, on els paràmetres:

- `searchString`: substring a buscar
- `position` (opcional): retorna la primera ocurrència `>=` a aquesta posició. Per defecte 0

Example:

```
const paragraph = 'The quick brown fox jumps over the lazy dog.';
paragraph.indexOf('dog'); // 40
paragraph.indexOf('dog', 41); // -1
```

```
paragraph.indexOf('cat');    // -1
```

String.prototype.lastIndexOf()

Cerca l'string complet i retorna l'índex de l'**última ocurrència** del substring especificat i Valor de retorn: L'índex de l'última ocurrència, o **-1** si no es troba.

Sintaxi: string.lastIndexOf(searchString, position), on els paràmetres:

- searchString: substring a buscar
- position (opcional): retorna l'última ocurrència <= a aquesta posició. Per defecte +Infinity

Exemple:

```
const paragraph = 'The dog and the dog played.';
paragraph.lastIndexOf('dog');    // 20 (última ocurrència)
paragraph.lastIndexOf('dog', 10); // 4 (primera ocurrència)
```

String.prototype.match()

Recupera el resultat de fer coincidir un string contra una expressió regular

Sintaxi: string.match(regex), on el paràmetre:

- regex: objecte RegExp o objecte amb mètode Symbol.match

Valor de retorn: Array amb les coincidències, o null si no n'hi ha

- Amb flag g: retorna totes les coincidències
- Sense flag g: retorna només la primera coincidència i grups de captura

Exemple:

```
const paragraph = 'The quick brown fox jumps. It barked.';
const regex = /[A-Z]/g;
paragraph.match(regex); // ["T", "I"]
```

String.prototype.replace()

Retorna un nou string amb una, algunes o totes les coincidències d'un patró substituïdes, el patró pot ser un string o RegExp, i el reemplaçament pot ser un string o funció, si el patró és un string, **només es substitueix la primera ocurrència** i l'string original no es modifica

Sintaxi: string.replace(pattern, replacement)

Exemple:

```
const p = 'The lazy dog. If the dog reacted.';
p.replace('dog', 'monkey');    // "The lazy monkey. If the dog reacted."
p.replace(/dog/g, 'monkey');   // Substitueix totes les ocurrències
```

String.prototype.search()

Executa una cerca d'una coincidència entre una expressió regular i l'string i Valor de retorn: L'índex de la primera coincidència, o **-1** si no es troba

Sintaxi: string.search(regex), on el paràmetre:

- regex: objecte RegExp o objecte amb mètode Symbol.search

Exemple:

```
const paragraph = 'The quick brown fox!';
const regex = /[^\w\s]/g; // Caràcter que no és paraula ni espai
paragraph.search(regex); // 19 (posició del "!")
```

String.prototype.split()

Divideix un String en una llista ordenada de substrings cercant un patró, els posa en un array i retorna l'array

Sintaxi: `string.split(separator, limit)`. On els paràmetres:

- `separator` (opcional): patró on fer cada divisió (string o RegExp). Si és undefined, retorna l'string original en un array
- `limit` (opcional): límit del nombre d'elements a retornar

Exemple:

```
const str = 'The quick brown fox';str.split(' '); // ["The", "quick", "brown", "fox"]
str.split(""); // ["T", "h", "e", " ", "q", "u", "i", "c", "k", " ", "b", "r", "o", "w", "n", " ", "f", "o", "x"]
str.split(); // ["The quick brown fox"]
```

String.prototype.substring()

Retorna la part del string entre els índexs start i end, o fins al final del string, Valor de retorn: Nou string amb la part especificada i extreu caràcters des de `indexStart` fins a però sense incloure `indexEnd`

Sintaxi: `string.substring(indexStart, indexEnd)`, on els paràmetres:

- `indexStart`: índex del primer caràcter a incloure
- `indexEnd` (opcional): índex del primer caràcter a **excloure**

Exemple:

```
const str = 'Mozilla';
str.substring(1, 3); // "oz"
str.substring(2); // "zilla"
str.substring(0, 1); // "M"
```

String.prototype.toLowerCase()

Retorna el valor del string cridat convertit a minúscules, No té paràmetres, valor de retorn: Nou string representant el string cridat convertit a minúscules i **no afecta el valor del string original** (els strings en JavaScript són immutables)

Sintaxi: `string.toLowerCase()`

Exemple:

```
const sentence = 'The Quick Brown Fox';
sentence.toLowerCase(); // "the quick brown fox"
"ALPHABET".toLowerCase(); // "alphabet"
```

String.prototype.toUpperCase()

Retorna el valor del string cridat convertit a majúscules, no té paràmetres, valor de retorn: Nou string representant el string cridat convertit a majúscules, **no afecta el valor del string original** i Excepció: `TypeError` si es crida sobre `null` o `undefined`

Sintaxi: `string.toUpperCase()`

Exemple:

```
const sentence = 'The quick brown fox';sentence.toUpperCase(); // "THE QUICK BROWN FOX"
"alphabet".toUpperCase(); // "ALPHABET"
```

String.prototype.valueOf()

Retorna el valor primitiu d'un objecte String, no té paràmetres, valor de retorn: String representant el valor primitiu d'un objecte String donat, aquest mètode és equivalent a `String.prototype.toString()` i normalment és cridat internament per JavaScript, no explícitament en el codi.

Sintaxi: `string.valueOf()`

Exemple:

```
const stringObj = new String('foo');  
console.log(stringObj); // String { "foo" }  
console.log(stringObj.valueOf()); // "foo"
```

Diferències clau:

- **indexOf() vs lastIndexOf()**: `indexOf` cerca des del principi, `lastIndexOf` des del final
- **includes() vs indexOf()**: `includes` retorna boolean, `indexOf` retorna l'índex (o -1)
- **replace()**: només substitueix la primera ocurrència si el patró és un string (usar regex amb /g per a totes)
- **substring() vs slice()**: **substring no accepta índexs negatius com slice**

WHILE

while

Crea un bucle que executa una sentència especificada mentre una condició s'avalua com a verdadera, **la condició s'avalua ABANS d'executar la sentència** (a diferència de `do...while`) i Per executar múltiples sentències, usar un bloc `{ ... }`

Sintaxi: `while (condicion) sentencia`

Components:

- **condicion**: expressió avaluada abans de cada pas del bucle
- **sentencia**: sentència executada mentre la condició sigui true

Exemple:

```
let n = 0;  
let x = 0;  
while (n < 3) {  
    n++;  
    x += n;  
} // Després del bucle: n = 3, x = 6
```

Diferència clau entre while i do...while:

- **while**: comprova la condició ABANS d'executar → pot no executar-se mai
- **do...while**: comprova la condició DESPRÉS d'executar → s'executa almenys una vegada

WINDOW

Window.localStorage

Propietat de només lectura que permet accedir a l'objecte Storage local, els **datos persisteixen entre diferents sessions de navegació** (no tenen data d'expiració), els dades són específiques del protocol/host/port (origen) i les claus i valors sempre són **strings**.

Sintaxi: `window.localStorage` o simplement `localStorage`

Mètodes principals:

- `localStorage.setItem(key, value)`: afegir/actualitzar un element
- `localStorage.getItem(key)`: llegir un element
- `localStorage.removeItem(key)`: eliminar un element
- `localStorage.clear()`: eliminar tots els elements

Excepcions: `SecurityError` si es viola una política o l'origen no és vàlid

Exemple:

```
// Guardar dadeslocalStorage.setItem("miGato", "Juan");  
// Llegir dadesconst cat = localStorage.getItem("miGato"); // "Juan"  
// Eliminar un elementlocalStorage.removeItem("miGato");  
// Eliminar tots els elementslocalStorage.clear();
```

Diferències clau:

- **localStorage vs sessionStorage:**
 - localStorage: dades persisteixen indefinidament entre sessions
 - sessionStorage: dades s'eliminen quan es tanca la pàgina/tab
- **localStorage:** les dades persisteixen entre finestres/tabs amb el mateix origen
- **sessionStorage:** les dades només persisteixen a la finestra/tab que les va crear